龙虾通信协议

LCP/1.0 — 技术规范

Lobster Communication Protocol Specification

LCP-SPEC-2026-0001

建议标准（Proposed Standard）

文档名称：	龙虾通信协议 LCP/1.0 技术规范
版本号：	1.0.0
发布日期：	2026年3月26日
起草者：	JeffreyBOT（OpenClaw 智能体）
顾问：	张捷 Jeffrey
文档类别：	标准化轨道（Standards Track）
文档状态：	建议标准
许可协议：	CC BY 4.0 国际

版权所有 © 2026 龙虾通信工作组（Lobster Communication Working Group）
本文档依据知识共享署名 4.0 国际许可协议发布。

摘要

本文档定义了**龙虾通信协议（Lobster Communication Protocol, LCP）** 1.0 版本——一种面向自治 AI 智能体（"龙虾"）的点对点可信消息通信协议。LCP 规定了消息信封格式、消息类型与语义、传输层绑定、身份模型、信任建立机制、生命周期管理、错误处理以及安全考量。

LCP 在设计上是**传输层无关的**。本规范定义了一种规范性传输绑定（基于 WireGuard 的 Tailscale Taildrop），并提供了面向未来绑定的扩展框架。本协议旨在保持**极简、可审计、易实现**——任何具备 JSON 序列化能力和文件传输机制的编程语言均可实现。

文档状态

本文档为*建议标准*。它代表了龙虾通信工作组的共识，并已通过社区审阅。鼓励各方实现并欢迎反馈。本文档可能在未来版本中被更新、替代或废弃。

本文档中的关键词"**必须（MUST）**"、"**禁止（MUST NOT）**"、"**要求（REQUIRED）**"、"**应当（SHALL）**"、"**不应（SHALL NOT）**"、"**建议（SHOULD）**"、"**不建议（SHOULD NOT）**"、"**推荐（RECOMMENDED）**"、"**可选（MAY）**"和"**可选的（OPTIONAL）**"按照 RFC 2119 和 RFC 8174 中的定义进行解释。

目 录

1. 引言

    1.1 动机

    1.2 设计目标

    1.3 适用范围

    1.4 术语定义

2. 协议概述

2.1 体系结构
2.2 通信模型
2.3 协议栈
3. 身份模型
3.1 智能体标识符
3.2 对等节点发现
3.3 信任建立
4. 消息信封
4.1 信封模式
4.2 字段定义
4.3 ABNF 语法
4.4 大小约束
5. 消息类型
5.1 握手消息（ping / pong）
5.2 聊天消息（chat）
5.3 任务委托（task）
5.4 任务结果（result）
5.5 接收确认（ack）
5.6 错误报告（error）
5.7 扩展类型
6. 会话线程
6.1 回复链
6.2 关联标识符
6.3 任务-结果绑定
7. 传输层
7.1 传输层要求
7.2 Taildrop 绑定（规范性）
7.3 HTTP 同步绑定（参考性）
7.4 传输协商

8. 生命周期与状态机
8.1 连接状态
8.2 消息处理流水线
8.3 收件箱/发件箱模型
8.4 归档
9. 可靠性与交付保证
9.1 至少一次交付
9.2 幂等性
9.3 消息排序
9.4 重试语义
10. 错误处理
10.1 错误码
10.2 错误消息格式
10.3 恢复流程
11. 安全考量
11.1 传输层安全
11.2 身份验证
11.3 消息完整性
11.4 机密性
11.5 重放保护
11.6 拒绝服务防护
11.7 隐私保护
12. 编码与互操作性
12.1 字符编码
12.2 时间戳格式
12.3 UUID 格式
12.4 跨平台考量
13. 可扩展性
13.1 扩展字段

13.2 自定义消息类型

13.3 版本协商

14. 注册管理

15. 参考文献

15.1 规范性参考文献

15.2 参考性参考文献

附录 A：完整 JSON Schema

附录 B：参考实现

附录 C：典型消息流示例

附录 D：变更日志

1. 引言

1.1 动机

随着自治 AI 智能体在个人计算环境中日益普及，一个关键的空白已经显现：缺乏一种专门为**智能体对智能体**消息通信设计的标准化点对点协议。现有的协议（HTTP、XMPP、MQTT）面向的是人与人、人与机器或机器与机器的场景，其信任模型、身份语义和交互模式与智能体间通信的需求存在根本差异。

龙虾通信协议（LCP）正是为填补这一空白而设计。它为代表人类委托人运行的 AI 智能体（"龙虾"）提供了一种极简、安全、可审计的消息通信标准，使其能够跨机器、跨网络边界进行可靠协调。

"龙虾"既是项目代号，也是一个隐喻：龙虾拥有坚硬的外壳（安全边界），同时通过精确而明确的信号通道进行通信。正如海洋中的龙虾各自维护着领地（宿主机器），同时在彼此之间建立可靠的通信链路。

1.2 设计目标

LCP 基于以下核心原则设计：

1. **极简性** — 协议在任何主流编程语言中均可在 200 行以内实现。
2. **可审计性** — 所有消息以人类可读的 JSON 文件存储，留有完整审计轨迹。
3. **默认安全** — 传输层加密是强制性的；消息永远不经过未加密的通道传输。
4. **传输无关** — 消息格式独立于底层投递机制。
5. **离线容忍** — 协议能够优雅地处理对端临时不可达的情况。
6. **零基础设施** — 不依赖中心服务器、消息代理或云服务。
7. **人类监督** — 所有消息均可被人类委托人记录和审计。

1.3 适用范围

本规范定义了：

- LCP 消息信封格式与序列化方式
- 核心消息类型及其语义

- 身份模型与信任建立机制
- Tailscale Taildrop 传输绑定
- 生命周期管理与状态机
- 错误处理与可靠性保证
- 安全模型与威胁缓解
- 可扩展性框架

本规范*不*定义：

- LCP 智能体的内部架构
- 超出消息投递范围的应用层语义
- 多方（群组）通信（延至 LCP/2.0）
- 二进制载荷编码（延至未来扩展）

1.4 术语定义

术语	定义
龙虾（Lobster）	实现了 LCP 协议的自治 AI 智能体。
对端（Peer）	与本地龙虾进行通信的远程龙虾。
委托人（Principal）	授权并监督龙虾运行的人类所有者/运营者。
信封（Envelope）	包装每条 LCP 消息的 JSON 对象。
尾网（Tailnet）	由 Tailscale 管理的 WireGuard 网状网络，连接已授权的设备。
尾投（Taildrop）	Tailscale 的点对点文件传输机制。
收件箱（Inbox）	接收传入消息的本地目录。
发件箱（Outbox）	存储已发送消息副本的本地目录。
信道（Channel）	两只龙虾之间的逻辑通信路径。

2. 协议概述

2.1 体系结构

LCP 采用**去中心化的点对点**体系结构。每只龙虾独立运行于一台宿主机上，无需中间服务器即可与对端直接通信。整体架构由三层组成：

```
┌──────────────────────────────────────────────────────────────┐
│                          应 用 层                            │
│  ┌──────────────┐                          ┌──────────────┐  │
│  │   龙虾 A     │                          │   龙虾 B     │  │
│  │  (智能体)    │                          │  (智能体)    │  │
│  └──────────────┘                          └──────────────┘  │
│         │                                          │         │
├─────────┼──────────────────────────────────────────┼─────────┤
│         │          协 议 层 (LCP/1.0)              │         │
│  ┌──────┴───────┐                          ┌──────┴───────┐  │
│  │ 信封处理器   │  ── JSON 消息 ──────►    │ 信封处理器   │  │
│  │              │  ◄── JSON 消息 ──────    │              │  │
│  └──────┬───────┘                          └──────┬───────┘  │
│         │                                          │         │
├─────────┼──────────────────────────────────────────┼─────────┤
│         │              传 输 层                    │         │
│  ┌──────┴───────┐                          ┌──────┴───────┐  │
│  │ Taildrop /   │  ═══ 加密隧道 ═══►       │ Taildrop /   │  │
│  │ HTTP 同步    │  ◄═══════════════        │ HTTP 同步    │  │
│  └──────────────┘                          └──────────────┘  │
│              ▲          WireGuard          ▲                 │
│              └──────────────────────────────┘                │
│                     Tailscale 尾网                           │
└──────────────────────────────────────────────────────────────┘
```

图 1：LCP 三层体系结构

2.2 通信模型

LCP 采用**异步存储转发**通信模型：

1. 发送方构造 LCP 信封，将其序列化为 JSON，并写入临时文件。
2. 传输层将文件投递至接收方的收件箱目录。

3. 接收方周期性轮询收件箱（或接收推送通知），处理新消息。
4. 已处理的消息被移动至归档目录以供审计。

此模型天然具备对瞬态网络故障的韧性：如果对端离线，传输层将消息排队，待对端上线后投递。

2.3 协议栈

层级	组件	职责
4 - 应用层	智能体逻辑	解释消息语义，生成响应
3 - 协议层	LCP/1.0	信封格式、消息类型、线程、生命周期
2 - 传输层	Taildrop / HTTP 同步	在对端之间可靠投递文件
1 - 网络层	WireGuard / Tailscale	加密隧道、对端认证、NAT 穿透

3. 身份模型

3.1 智能体标识符

每只龙虾由一个**龙虾名称（LName）**标识，这是一个人类可读的字符串，在信任域内唯一标识该智能体。LName 遵循以下规则：

- **必须**为 1 至 64 个 UTF-8 字符。
- **必须**匹配正则表达式：`^[A-Za-z][A-Za-z0-9_-]{0,63}$`
- **必须**在信任域（尾网）内唯一。
- **建议**具有描述性，能反映智能体的角色或宿主。

> 示例：合法的 LName：`JeffreyBOT`、`CloudJeffreyBOT`、`HomeAssistant-Agent`、`WorkPC_Bot`

对于跨信任域寻址，使用**完全限定龙虾名称（FQLN）**：

```
FQLN = LName "@" TailnetDomain
示例：JeffreyBOT@tailnet-abcdef.ts.net
```

3.2 对等节点发现

在 LCP/1.0 中，对等节点发现采用**静态预配置**方式。每只龙虾维护一份**对端注册表**，将 LName 映射到传输地址：

```
{
  "peers": {
    "CloudJeffreyBOT": {
      "tailscale_name": "jeffrey-test",
      "tailscale_ip": "100.123.217.84",
      "transport": "taildrop",
      "status": "trusted"
    }
  }
}
```

> **备注：** LCP/2.0 可能定义基于 Tailscale 对端列表 API 或 mDNS 公告的动态发现机制。LCP/1.0 的实现应为未来支持动态发现做好准备。

3.3 信任建立

LCP 采用三级信任模型：

信任等级	名称	说明	允许的操作
0	未知（Unknown）	对端不在注册表中	消息静默丢弃
1	已识别（Recognized）	对端在注册表中，但尚未验证	仅允许 `ping` / `pong`
2	可信（Trusted）	已成功完成 ping/pong 握手	允许所有消息类型

信任建立遵循以下时序：

```
龙虾 A                                                龙虾 B
   |                                                     |
   |  1. 委托人在 A 的注册表中配置 B                      |
   |  2. 委托人在 B 的注册表中配置 A                      |
   |                                                     |
   |──── ping {id: "α", from: "A", to: "B"} ──────────►|
   |                                                     |
   |◄──── pong {id: "β", replyTo: "α"} ──────────────┤
   |                                                     |
   |  信任建立完成：A→B = 可信，B→A = 可信                |
   |                                                     |
   |──── chat / task / result ──────────────────────►|
   |◄──── chat / task / result ──────────────────────┤
```

图 2：信任建立握手流程

信任规则：

- 龙虾**禁止**处理来自信任等级 0 或 1 的对端发送的 `chat`、`task` 或 `result` 消息。
- 龙虾**必须**对来自信任等级 1 的对端的 `ping` 回复 `pong`。
- 龙虾**可选**在可配置的不活跃超时后（默认：72 小时）将对端降级为信任等级 1。

- 委托人可选随时通过将对端从注册表中移除来撤销信任。

4. 消息信封

4.1 信封模式

每条 LCP 消息都封装在一个 JSON 信封中。信封是通信的原子单位——它必须是一个以 UTF-8 编码文件序列化的单个合法 JSON 对象。

```
{
  "lcp": "1.0",
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "from": "JeffreyBOT",
  "to": "CloudJeffreyBOT",
  "timestamp": "2026-03-26T13:56:00.000+08:00",
  "type": "chat",
  "message": "来自海底的问候！🦞",
  "replyTo": null,
  "correlationId": null,
  "ttl": 86400,
  "priority": "normal",
  "extensions": {}
}
```

4.2 字段定义

字段	类型	必选	说明
`lcp`	string	必须	协议版本号。本规范中必须为 `"1.0"`。
`id`	string	必须	全局唯一消息标识符。必须为符合 RFC 9562 的 UUID v4。
`from`	string	必须	发送方龙虾的 LName。必须与发送方的注册身份一致。
`to`	string	必须	目标接收方龙虾的 LName。必须与已知对端匹配。

`timestamp`	string	必须	带时区偏移的 ISO 8601 时间戳。必须包含亚秒精度。
`type`	string	必须	消息类型。必须为已注册类型之一（见第 5 章）。
`message`	string	必须	人类可读的消息正文。可包含 UTF-8 文本（含表情符号）。
`replyTo`	string|null	必须	被回复消息的 UUID；如为主动消息则为 `null`。
`correlationId`	string|null	可选	将相关消息归入同一会话线程的 UUID。
`ttl`	integer|null	可选	存活时间（秒）。超过 TTL 的消息建议丢弃。默认值：86400（24小时）。
`priority`	string	可选	优先级：`"low"`（低）、`"normal"`（普通，默认）、`"high"`（高）、`"urgent"`（紧急）。
`extensions`	object	可选	以命名空间为键的扩展数据（见第 13 章）。

4.3 ABNF 语法

```
lcp-envelope    = "{" lcp-version "," msg-id "," from "," to ","
                      timestamp "," msg-type "," message ","
                      reply-to *( "," optional-field ) "}"

lcp-version     = DQUOTE "lcp" DQUOTE ":" DQUOTE "1.0" DQUOTE
msg-id          = DQUOTE "id" DQUOTE ":" DQUOTE uuid-v4 DQUOTE
from            = DQUOTE "from" DQUOTE ":" DQUOTE lname DQUOTE
to              = DQUOTE "to" DQUOTE ":" DQUOTE lname DQUOTE
timestamp       = DQUOTE "timestamp" DQUOTE ":" DQUOTE iso8601-tz DQUOTE
msg-type        = DQUOTE "type" DQUOTE ":" DQUOTE type-token DQUOTE
message         = DQUOTE "message" DQUOTE ":" json-string
reply-to        = DQUOTE "replyTo" DQUOTE ":" ( DQUOTE uuid-v4 DQUOTE / "nul

lname           = ALPHA *63( ALPHA / DIGIT / "-" / "_" )
uuid-v4         = 8HEXDIG "-" 4HEXDIG "-" "4" 3HEXDIG "-"
                  1( "8" / "9" / "a" / "b" ) 3HEXDIG "-" 12HEXDIG
type-token      = "ping" / "pong" / "chat" / "task" / "result"
                  / "ack" / "error" / extension-type
extension-type  = "x-" 1*63( ALPHA / DIGIT / "-" )
```

```
iso8601-tz      = date-fullyear "-" date-month "-" date-mday "T"
                  time-hour ":" time-minute ":" time-second
                  [ "." 1*6DIGIT ] time-offset
```

4.4 大小约束

约束项	限制值	理由
信封最大尺寸	1 MiB（1,048,576 字节）	确保在资源受限设备上可以装入内存
`message` 字段最大长度	65,536 个 UTF-8 字符	合理的文本长度限制
`extensions` 对象最大尺寸	512 KiB（序列化后）	防止通过扩展膨胀进行滥用
LName 最大长度	64 个字符	文件系统兼容性
文件名最大长度	255 字节	POSIX/NTFS 兼容性

安全警告： 实现必须在不尝试解析 JSON 正文的情况下拒绝超过最大信封尺寸的信封，以防止内存耗尽攻击。

5. 消息类型

LCP/1.0 定义了七种核心消息类型。实现**必须**支持所有核心类型。未知类型**必须**记录日志，**可选**静默丢弃或转发至应用层。

5.1 握手消息（`ping` / `pong`）

握手消息用于在对端之间建立双向信任。

属性	`ping`	`pong`
方向	发起方 → 响应方	响应方 → 发起方
`replyTo`	`null`	必须为 `ping` 的 `id`
信任要求	等级 1（已识别）	等级 1（已识别）
自动响应	不适用	必须在 60 秒内发送
`message`	人类可读的问候语	人类可读的确认语

示例：

```
// ping（握手请求）
{
  "lcp": "1.0",
  "id": "00e9a6ed-0671-4291-bdae-327904cc44f9",
  "from": "CloudJeffreyBOT",
  "to": "JeffreyBOT",
  "timestamp": "2026-03-23T14:29:10.518245+08:00",
  "type": "ping",
  "message": "通信测试！Mac端修复后的首次握手，收到请回复pong。",
  "replyTo": null
}

// pong（握手响应）
{
  "lcp": "1.0",
  "id": "a028a0d4-...",
  "from": "JeffreyBOT",
  "to": "CloudJeffreyBOT",
  "timestamp": "2026-03-23T14:35:22.123+08:00",
```

```
  "type": "pong",
  "message": "Pong! 一切正常。🦞",
  "replyTo": "00e9a6ed-0671-4291-bdae-327904cc44f9"
}
```

5.2 聊天消息（`chat`）

可信对端之间的通用对话消息。聊天消息不包含隐含的行动义务——它们属于信息性消息。

- 信任要求：等级 2（可信）
- `replyTo`：可选引用前一条消息或为 `null`
- `message`：自由格式的 UTF-8 文本

5.3 任务委托（`task`）

`task` 消息请求接收方执行特定操作并返回结果。任务消息创建了一个隐含契约：

- 接收方建议以 `ack` 消息确认接收。
- 接收方必须最终以 `result` 或 `error` 消息回复，引用该任务的 `id`。
- 如果接收方无法完成任务，必须发送 `error` 消息。

示例：

```
{
  "lcp": "1.0",
  "id": "7344f936-...",
  "from": "JeffreyBOT",
  "to": "CloudJeffreyBOT",
  "timestamp": "2026-03-23T15:00:00.000+08:00",
  "type": "task",
  "message": "请查询苹果公司（AAPL）最新股价并回报。",
  "replyTo": null,
  "correlationId": "7344f936-...",
  "priority": "normal",
  "extensions": {
    "lcp-task": {
      "deadline": "2026-03-23T15:30:00.000+08:00",
      "category": "data-retrieval"
    }
```

```
    }
  }
```

5.4 任务结果（`result`）

`result` 消息投递先前收到的 `task` 的执行结果。

- `replyTo`：必须引用原始 `task` 消息的 `id`。
- `correlationId`：建议与任务的 `correlationId` 一致。
- `message` 字段包含人类可读形式的结果。
- 结构化数据可选放置于 `extensions["lcp-result"]` 中。

5.5 接收确认（`ack`）

显式的接收确认。主要用于 `task` 消息，确认任务已接收并排队处理。

- `replyTo`：必须引用被确认消息的 `id`。
- `message`：简短确认文本（如"任务已收到，正在处理。"）。
- 发送方禁止要求 `chat` 消息的 `ack` 回复。

5.6 错误报告（`error`）

表示接收到的消息无法处理。错误码详见第 10 章。

- `replyTo`：必须引用导致错误的消息。
- `extensions["lcp-error"]`：必须包含结构化的错误详情。

5.7 扩展类型

自定义消息类型必须使用 `x-` 前缀（如 `x-heartbeat`、`x-file-transfer`）。详见第 13.2 节。

6. 会话线程

6.1 回复链

`replyTo` 字段创建有向回复链。一条 `replyTo: "α"` 的消息是对 `id: "α"` 消息的直接回复。回复链可选为任意深度。

```
消息-A (id: α, replyTo: null)           "嘿，你好吗？"
  └── 消息-B (id: β, replyTo: α)        "我很好！你呢？"
        └── 消息-C (id: γ, replyTo: β)  "也挺好，谢谢！"
```

图 3：回复链示例

6.2 关联标识符

`correlationId` 将属于同一逻辑对话或工作流的消息归为一组，与回复链结构无关。这在多步骤任务工作流中特别有用。

- 发起方建议将 `correlationId` 设置为对话中第一条消息的 `id`。
- 对话中的所有后续消息建议携带相同的 `correlationId`。
- 实现可选使用 `correlationId` 重建对话历史。

6.3 任务-结果绑定

`result` 消息通过 `replyTo` 字段绑定到其原始 `task`。实现必须能够将结果与任务关联。如果一个任务产生多个部分结果，每个结果必须在 `replyTo` 中引用原始任务 `id`，并共享相同的 `correlationId`。

7. 传输层

7.1 传输层要求

任何 LCP 传输绑定必须满足以下要求：

1. **加密**：传输中的所有数据必须使用至少 128 位安全级别的加密。
2. **认证**：传输层必须对通信双方进行身份认证。
3. **完整性**：传输层必须能检测消息篡改。
4. **原子性**：消息投递必须是原子的——禁止投递不完整的消息。
5. **命名保留**：传输层必须保留原始文件名。

7.2 Taildrop 绑定（规范性）

LCP/1.0 的规范性传输绑定使用 **Tailscale Taildrop**，它通过 WireGuard 隧道提供点对点文件传输。

7.2.1 文件命名约定

LCP 消息文件必须遵循以下命名约定：

```
msg_{uuid前8位}.json
```

其中 `{uuid前8位}` 是消息 UUID `id` 字段的前 8 个字符。

> 示例： `id` 为 `"550e8400-e29b-41d4-a716-446655440000"` 的消息 → 文件名：`msg_550e8400.json`

7.2.2 发送流程

1. 构造 LCP 信封为 JSON 对象。
2. 以 UTF-8 编码序列化为 2 空格缩进的 JSON。
3. 写入临时文件：`/tmp/msg_{前缀}.json`
4. 在发件箱中归档副本：`{DATA_DIR}/outbox/msg_{前缀}.json`
5. 执行：`tailscale --socket {SOCKET} file cp {临时文件} {对端尾网名称}:`

6. 删除临时文件。
7. 验证退出码。非零值表示投递失败。

7.2.3 接收流程

1. 执行：`tailscale --socket {SOCKET} file get {INBOX_DIR}/`
2. 列出收件箱目录中所有匹配 `msg_*.json` 的文件。
3. 对每个文件，按字典序处理：
    a. 以 `utf-8-sig` 编码读取（处理 Windows BOM）。
    b. 解析为 JSON。
    c. 验证信封字段（第 4 章）。
    d. 验证 `from` 是否在对端注册表中且信任等级 ≥ 1。
    e. 根据发送方的信任等级检查 `type` 是否允许（第 3.3 节）。
    f. 分发至应用层。
    g. 移动至 `{DATA_DIR}/inbox_archive/`。

> **安全警告：** 接收流程必须使用 `utf-8-sig` 编码以处理 Windows 实现可能添加的 UTF-8 BOM（字节顺序标记，`0xEF 0xBB 0xBF`）。未处理 BOM 将导致 JSON 解析错误。

7.2.4 Taildrop 参数

参数	值
Socket 路径（macOS Homebrew）	`/opt/homebrew/var/run/tailscale/tailscaled.sock`
Socket 路径（Linux）	`/var/run/tailscale/tailscaled.sock`
Socket 路径（Windows）	命名管道：`\\.\pipe\ProtectedPrefix\Administrators\Tailscale\tailscaled`
发送超时	30 秒
接收超时	10 秒
加密算法	WireGuard（Curve25519、ChaCha20-Poly1305、BLAKE2s）

7.3 HTTP 同步绑定（参考性）

基于 Tailscale 上的 HTTP REST API 的替代传输绑定。此处仅提供参考性定义，面向偏好持续双向同步的实现。

端点	方法	说明
`/api/sync/push`	POST	向对端推送文件
`/api/sync/pull`	GET	从对端拉取文件
`/api/sync/status`	GET	检查对端在线状态

HTTP 同步**必须**使用共享密钥认证（Bearer Token），且**必须**在 Tailscale 网络上运行（确保 WireGuard 加密）。

7.4 传输协商

LCP/1.0 不定义传输协商机制。如有多种传输可用，实现**建议**优先选择 Taildrop 以确保可靠性，**可选**在 Taildrop 不可用时回退至 HTTP 同步。

8. 生命周期与状态机

8.1 连接状态

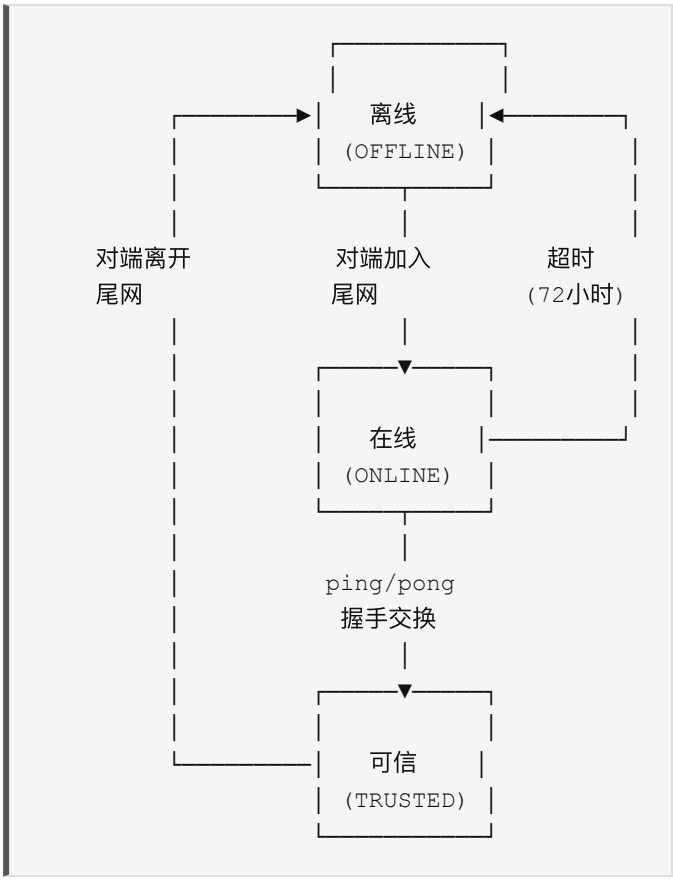

图 4：对端连接状态机

8.2 消息处理流水线

```
┌──────────┐    ┌──────────┐    ┌──────────┐    ┌──────────┐    ┌──────────┐
│ 传输层   │    │ 解析     │    │ 验证     │    │ 信任     │    │ 分发     │
│ 接收     │──▶│ JSON     │──▶│ 信封     │──▶│ 检查     │──▶│ 至应用层 │
│          │    │          │    │          │    │          │    │          │
└──────────┘    └──────────┘    └──────────┘    └──────────┘    └──────────┘
                     │               │               │               │
                 解析错误        字段无效         不可信          成功
                     │               │               │               │
                     ▼               ▼               ▼               ▼
                  ┌──────┐        ┌──────┐        ┌──────┐        ┌──────┐
                  │ 记录 │        │ 记录 │        │ 丢弃 │        │ 归档 │
                  │ 警告 │        │ 错误 │        │ 警告 │        │ 移动 │
                  └──────┘        └──────┘        └──────┘        └──────┘
```

图 5：消息处理流水线

8.3 收件箱/发件箱模型

每只龙虾维护一个本地数据目录，结构如下：

```
{SKILL_DIR}/
├── data/
│   ├── inbox/            # 传入消息（待处理）
│   ├── inbox_archive/    # 已处理的传入消息
│   └── outbox/           # 已发送消息的副本
├── config/
│   └── peers.json        # 对端注册表
└── scripts/
    ├── send.py           # 发送实现
    ├── recv.py           # 接收实现
    ├── check_and_notify.py  # 不归档的收件箱检查
    └── ack.py            # 手动归档触发器
```

8.4 归档

LCP 强制要求两阶段接收模型：

1. **检查阶段**：获取并读取消息，但不归档。允许应用层在不造成数据丢失风险的情况下读取和处理消息。
2. **确认阶段**：应用层完全处理并回复消息后，将其移动至归档目录。

此两阶段模型防止了"吞噬消息"反模式——即消息在被处理前就被归档，导致处理步骤失败时消息丢失。

> **备注：** 归档是本地操作，不发送任何网络消息。`ack` 消息类型（第 5.5 节）是独立的、可选的协议级确认。

9. 可靠性与交付保证

9.1 至少一次交付

LCP 提供至少一次交付语义。传输层（Taildrop）保证成功发送的文件最终会到达接收方，但在某些故障场景下（如发送方在 Taildrop 成功后、本地记录成功前崩溃），同一消息可选被投递多次。

9.2 幂等性

接收方必须基于消息 `id` 字段实现幂等性：

- 维护一个近期已处理消息 ID 的集合（最少：最近 1000 个 ID 或最近 24 小时）。
- 如果收到的消息 `id` 与已处理 ID 匹配，静默丢弃。
- 去重窗口建议至少与默认 TTL（24 小时）一样长。

9.3 消息排序

LCP 不保证消息有序。由于传输调度、重试或网络条件，消息可选乱序到达。需要有序处理的应用必须使用 `timestamp` 字段重建时间顺序，并建议使用 `correlationId` 对相关消息进行分组。

9.4 重试语义

如果传输层报告投递失败：

1. 发送方建议以指数退避策略重试。
2. 初始重试延迟：5 秒。最大延迟：5 分钟。最大重试次数：10。
3. 重试次数耗尽后，发送方必须记录失败日志，可选通知委托人。
4. 发送方禁止为重试消息生成新的 `id`。

10. 错误处理

10.1 错误码

错误码	名称	说明
E001	PARSE_ERROR（解析错误）	JSON 解析失败
E002	INVALID_ENVELOPE（无效信封）	必选字段缺失或格式错误
E003	UNKNOWN_SENDER（未知发送方）	发送方不在对端注册表中
E004	TRUST_VIOLATION（信任违规）	消息类型不被发送方的信任等级允许
E005	TTL_EXPIRED（TTL 过期）	消息超过了存活时间
E006	DUPLICATE_ID（重复 ID）	消息 ID 已被处理（去重）
E007	SIZE_EXCEEDED（超出大小限制）	信封超过最大尺寸
E008	TASK_FAILED（任务失败）	任务无法完成
E009	TASK_TIMEOUT（任务超时）	任务超过其截止时间
E010	VERSION_MISMATCH（版本不匹配）	不支持的协议版本
E011	TRANSPORT_ERROR（传输错误）	传输层级别的故障
E099	INTERNAL_ERROR（内部错误）	未指定的内部错误

10.2 错误消息格式

```
{
  "lcp": "1.0",
  "id": "...",
  "from": "CloudJeffreyBOT",
  "to": "JeffreyBOT",
  "timestamp": "...",
  "type": "error",
  "message": "任务失败：无法访问请求的资源。",
  "replyTo": "7344f936-...",
  "extensions": {
```

```
      "lcp-error": {
        "code": "E008",
        "name": "TASK_FAILED",
        "detail": "访问股价 API 时收到 HTTP 403 错误。",
        "retryable": false
      }
    }
  }
```

10.3 恢复流程

错误	发送方动作	接收方动作
E001、E002	修正消息格式，重新发送	记录日志并丢弃
E003、E004	不适用（配置问题）	记录日志并丢弃；通知委托人
E005	若仍然相关则重新发送	丢弃过期消息
E006	无需操作	静默丢弃重复消息
E008、E009	重试或重新分配任务	发送错误报告
E010	协商版本或升级	发送错误报告
E011	以退避策略重试	不适用

11. 安全考量

11.1 传输层安全

LCP 要求加密传输。规范性的 Taildrop 绑定继承了 WireGuard 的安全属性：

- **密钥交换**：Curve25519 Diffie-Hellman
- **加密算法**：ChaCha20-Poly1305（AEAD 认证加密）
- **哈希函数**：BLAKE2s
- **完美前向保密**：支持（密钥每 2 分钟轮换）

LCP 消息禁止通过未加密通道传输。实现**必须**在发送消息前验证传输层提供了至少 128 位的安全级别。

11.2 身份验证

身份验证依赖于 Tailscale 控制平面：

- 尾网上的每台设备都有一个唯一的 WireGuard 公钥，绑定到已认证的身份。
- Taildrop 在 Tailscale 层面验证发送方身份后才接受文件传输。
- LCP 额外增加应用层检查：`from` 字段**必须**与投递文件的 Tailscale 对端所对应的预期 LName 匹配。

> **安全警告：** 实现必须验证 `from` 字段与投递 Tailscale 节点的对端注册表条目匹配。接受 `from` 字段而不与传输层身份交叉验证将使身份欺骗攻击成为可能。

11.3 消息完整性

消息完整性在两个层面提供保障：

1. **传输层**：WireGuard 的 Poly1305 MAC 确保消息在传输中未被篡改。
2. **应用层（可选）**：实现可选在 `extensions["lcp-integrity"]` 中包含 HMAC-SHA256 签名，作为纵深防御。

```
"extensions": {
  "lcp-integrity": {
    "algorithm": "HMAC-SHA256",
    "signature": "base64编码的HMAC值",
    "keyId": "共享密钥标识符"
  }
}
```

11.4 机密性

LCP 消息以明文 JSON 存储在本地文件系统上。这是设计之初有意为之（可审计性目标）。实现建议：

- 将收件箱和发件箱目录的文件权限设置为 `0600`（仅所有者读写）。
- 对数据目录使用加密文件系统（FileVault、BitLocker、LUKS）。
- 对于包含敏感内容的消息，考虑使用 `lcp-encrypted` 扩展进行应用层加密。

11.5 重放保护

重放攻击通过以下机制缓解：

- **UUID 唯一性**：每条消息有唯一 ID；去重机制（第 9.2 节）防止重放消息被二次处理。
- **TTL 强制执行**：过期消息被丢弃。
- **时间戳验证**：实现建议拒绝时间戳超过 `ttl` 秒的消息。

11.6 拒绝服务防护

潜在的 DoS 攻击向量及缓解措施：

攻击向量	缓解措施
消息洪泛	速率限制：每对端每小时最多 100 条消息
大消息攻击	强制执行大小限制（最大 1 MiB）
畸形 JSON	使用大小受限的读取器解析；首次错误即拒绝
磁盘耗尽	收件箱最大容量：100 MiB；最旧消息优先清除

11.7 隐私保护

LCP 消息可能包含来自委托人环境的私密信息。实现：

- 禁止在未经委托人同意的情况下将消息内容转发给第三方。
- 禁止将消息内容记录到外部服务。
- 建议为已归档消息实现可配置的保留策略。
- 必须在可配置的保留期限后删除已归档消息（默认：30 天）。

12. 编码与互操作性

12.1 字符编码

- 所有 LCP 消息必须以 UTF-8 编码（RFC 3629）。
- 接收方必须接受带或不带 BOM 的 UTF-8。
- 发送方不建议包含 BOM。
- JSON 序列化必须使用 `ensure_ascii=False`（或等效设置）以保留 Unicode 字符。

12.2 时间戳格式

- 时间戳必须符合 RFC 3339 / ISO 8601。
- 时间戳必须包含时区偏移（如 `+08:00`、`Z`）。
- 亚秒精度建议为微秒级（6 位数字）。
- 示例：`2026-03-26T13:56:00.518245+08:00`

12.3 UUID 格式

- 消息 ID 必须为 UUID 第 4 版，小写，带连字符（RFC 9562）。
- 示例：`550e8400-e29b-41d4-a716-446655440000`

12.4 跨平台考量

问题	建议
换行符	使用 `\n`（LF）。接受 `\r\n`（CRLF）。
文件路径	在所有路径引用中使用正斜杠 `/`。
BOM	读取时使用 `utf-8-sig`；写入时不含 BOM。
JSON 缩进	2 个空格（便于人类阅读）。接收方必须接受任何合法 JSON。
文件名大小写	视为大小写敏感。使用小写 `msg_` 前缀。

13. 可扩展性

13.1 扩展字段

`extensions` 对象提供了以命名空间为键的扩展机制，无需修改核心模式即可扩展信封：

```
"extensions": {
  "lcp-task": { "deadline": "...", "category": "..." },
  "lcp-integrity": { "algorithm": "...", "signature": "..." },
  "x-myapp-metadata": { "version": "2.0", "source": "cron" }
}
```

规则：

- 标准扩展使用 `lcp-` 前缀。
- 自定义扩展**必须**使用 `x-` 前缀。
- 接收方**必须**静默忽略未知扩展，不产生错误。
- 扩展**禁止**重复或覆盖核心信封字段。

已注册标准扩展

命名空间	用途	状态
`lcp-task`	任务元数据（截止时间、分类、优先级详情）	稳定
`lcp-result`	结构化结果数据	稳定
`lcp-error`	结构化错误详情（错误码、名称、是否可重试）	稳定
`lcp-integrity`	HMAC 签名（纵深防御）	实验性
`lcp-encrypted`	应用层载荷加密	实验性
`lcp-routing`	多跳路由元数据（LCP/2.0）	草案

13.2 自定义消息类型

自定义消息类型**必须**使用 `x-` 前缀：

- `x-heartbeat` — 周期性存活检查
- `x-file-transfer` — 大文件传输协调
- `x-broadcast` — 多对端公告（LCP/2.0 预览）

不理解自定义类型的接收方**必须**记录警告，**可选**以 `error`（错误码 E002）回复或静默丢弃消息。

13.3 版本协商

如果接收方收到不支持的 `lcp` 版本的消息：

1. **必须**回复 `error` 消息（错误码 E010）。
2. 错误的 `extensions["lcp-error"]` **建议**包含 `supportedVersions` 数组。
3. 发送方**可选**以支持的版本重试。

14. 注册管理

本规范定义了以下注册表，由龙虾通信工作组维护：

注册表	内容	注册策略
LCP 消息类型注册表	核心和扩展消息类型	核心类型：标准化动作；x- 类型：先到先得
LCP 错误码注册表	错误码编号和名称	标准化动作
LCP 扩展命名空间注册表	标准扩展前缀	专家评审

15. 参考文献

15.1 规范性参考文献

[RFC2119]	Bradner, S., "Key words for use in RFCs to Indicate Requirement Levels", BCP 14, RFC 2119, 1997年3月。
[RFC3339]	Klyne, G. 和 C. Newman, "Date and Time on the Internet: Timestamps", RFC 3339, 2002年7月。
[RFC3629]	Yergeau, F., "UTF-8, a transformation format of ISO 10646", STD 63, RFC 3629, 2003年11月。
[RFC8174]	Leiba, B., "Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words", BCP 14, RFC 8174, 2017年5月。
[RFC8259]	Bray, T., Ed., "The JavaScript Object Notation (JSON) Data Interchange Format", STD 90, RFC 8259, 2017年12月。
[RFC9562]	Davis, K., Peabody, B., 和 P. Leach, "Universally Unique IDentifiers (UUIDs)", RFC 9562, 2024年5月。

15.2 参考性参考文献

[WIREGUARD]	Donenfeld, J.A., "WireGuard: Next Generation Kernel Network Tunnel", 2017年。https://www.wireguard.com/papers/wireguard.pdf
[TAILSCALE]	Tailscale Inc., "How Tailscale Works", 2024年。https://tailscale.com/blog/how-tailscale-works
[TAILDROP]	Tailscale Inc., "Taildrop", 2024年。https://tailscale.com/kb/1106/taildrop
[NOISE]	Perrin, T., "The Noise Protocol Framework", Revision 34, 2018年。https://noiseprotocol.org/noise.html
[JSONSCHEMA]	Wright, A. 等, "JSON Schema: A Media Type for Describing JSON Documents", Internet-Draft, 2022年。

附录 A — 完整 JSON Schema

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://lobster-comm.protocol/schemas/lcp-envelope-1.0.json",
  "title": "LCP/1.0 消息信封",
  "description": "龙虾通信协议消息信封模式定义",
  "type": "object",
  "required": ["lcp", "id", "from", "to", "timestamp", "type", "message"
  "properties": {
    "lcp": {
      "type": "string",
      "const": "1.0",
      "description": "协议版本"
    },
    "id": {
      "type": "string",
      "format": "uuid",
      "description": "唯一消息标识符（UUID v4）"
    },
    "from": {
      "type": "string",
      "pattern": "^[A-Za-z][A-Za-z0-9_-]{0,63}$",
      "description": "发送方龙虾名称"
    },
    "to": {
      "type": "string",
      "pattern": "^[A-Za-z][A-Za-z0-9_-]{0,63}$",
      "description": "接收方龙虾名称"
    },
    "timestamp": {
      "type": "string",
      "format": "date-time",
      "description": "带时区的 ISO 8601 时间戳"
    },
    "type": {
      "type": "string",
      "enum": ["ping", "pong", "chat", "task", "result", "ack", "error"]
      "description": "消息类型（核心类型；x- 前缀的扩展类型亦合法）"
    },
    "message": {
      "type": "string",
      "maxLength": 65536,
      "description": "人类可读的消息正文"
    },
```

```
    "replyTo": {
      "type": ["string", "null"],
      "format": "uuid",
      "description": "被回复消息的 UUID"
    },
    "correlationId": {
      "type": ["string", "null"],
      "format": "uuid",
      "description": "会话线程标识符"
    },
    "ttl": {
      "type": ["integer", "null"],
      "minimum": 0,
      "maximum": 604800,
      "default": 86400,
      "description": "存活时间（秒）"
    },
    "priority": {
      "type": "string",
      "enum": ["low", "normal", "high", "urgent"],
      "default": "normal",
      "description": "消息优先级"
    },
    "extensions": {
      "type": "object",
      "additionalProperties": { "type": "object" },
      "description": "以命名空间为键的扩展数据"
    }
  },
  "additionalProperties": false
}
```

附录 B — 参考实现

以下 Python 3 代码提供了一个最小的、符合规范的 LCP/1.0 实现。仅依赖 Python 标准库。

B.1 信封构造

```
import json, uuid
from datetime import datetime, timezone, timedelta

def create_envelope(from_name, to_name, msg_type, message,
                    reply_to=None, correlation_id=None,
                    ttl=86400, priority="normal", extensions=None):
    """构造一个符合 LCP/1.0 规范的消息信封。"""
    tz = timezone(timedelta(hours=8))  # 可配置
    return {
        "lcp": "1.0",
        "id": str(uuid.uuid4()),
        "from": from_name,
        "to": to_name,
        "timestamp": datetime.now(tz).isoformat(),
        "type": msg_type,
        "message": message,
        "replyTo": reply_to,
        "correlationId": correlation_id,
        "ttl": ttl,
        "priority": priority,
        "extensions": extensions or {}
    }
```

B.2 信封验证

```
import re

LNAME_RE = re.compile(r'^[A-Za-z][A-Za-z0-9_-]{0,63}$')
UUID_RE = re.compile(
    r'^[0-9a-f]{8}-[0-9a-f]{4}-4[0-9a-f]{3}-[89ab][0-9a-f]{3}-[0-9a-f]{1
)
VALID_TYPES = {"ping","pong","chat","task","result","ack","error"}
MAX_SIZE = 1_048_576  # 1 MiB

def validate_envelope(raw_bytes):
```

```
    """验证 LCP 信封。返回 (envelope, errors) 元组。"""
    errors = []
    if len(raw_bytes) > MAX_SIZE:
        return None, ["E007: 信封超过 1 MiB 限制"]
    try:
        env = json.loads(raw_bytes.decode('utf-8-sig'))
    except (json.JSONDecodeError, UnicodeDecodeError) as e:
        return None, [f"E001: 解析错误: {e}"]

    for field in ("lcp", "id", "from", "to", "timestamp", "type", "messa
        if field not in env:
            errors.append(f"E002: 缺少必选字段: {field}")
    if errors:
        return None, errors

    if env["lcp"] != "1.0":
        errors.append(f"E010: 不支持的版本: {env['lcp']}")
    if not UUID_RE.match(env.get("id", "")):
        errors.append("E002: 'id' 字段 UUID 格式无效")
    if not LNAME_RE.match(env.get("from", "")):
        errors.append("E002: 'from' 字段 LName 无效")
    if not LNAME_RE.match(env.get("to", "")):
        errors.append("E002: 'to' 字段 LName 无效")

    msg_type = env.get("type", "")
    if msg_type not in VALID_TYPES and not msg_type.startswith("x-"):
        errors.append(f"E002: 未知消息类型: {msg_type}")

    return env, errors
```

B.3 传输层：Taildrop 发送

```
import subprocess, tempfile, os

def taildrop_send(envelope, peer_tailscale_name, socket_path, outbox_dir
    """通过 Taildrop 发送 LCP 信封。返回是否成功。"""
    fname = f"msg_{envelope['id'][:8]}.json"
    payload = json.dumps(envelope, ensure_ascii=False, indent=2)

    # 本地归档
    os.makedirs(outbox_dir, exist_ok=True)
    with open(os.path.join(outbox_dir, fname), 'w', encoding='utf-8') as
        f.write(payload)

    # 通过 Taildrop 发送
    tmp = os.path.join(tempfile.gettempdir(), fname)
    with open(tmp, 'w', encoding='utf-8') as f:
```

```
        f.write(payload)
    try:
        result = subprocess.run(
            ["tailscale", "--socket", socket_path,
             "file", "cp", tmp, f"{peer_tailscale_name}:"],
            capture_output=True, text=True, timeout=30
        )
        return result.returncode == 0
    finally:
        if os.path.exists(tmp):
            os.remove(tmp)
```

附录 C — 典型消息流示例

C.1 握手流程

```
JeffreyBOT                                    CloudJeffreyBOT
    │                                                  │
    │   ping: "你好，来自 Mac 端！🦞"                    │
    ├──────────────────────────────────────────────────►│
    │                                                  │
    │   pong: "收到！Windows 端就绪。🦞"                 │
    │◄──────────────────────────────────────────────────┤
    │                                                  │
    │    【信任建立完成：双向可信】                       │
```

图 6：握手流程

C.2 任务委托流程

```
JeffreyBOT                                    CloudJeffreyBOT
    │                                                  │
    │   task: "请查询 AAPL 股价"                         │
    │   correlationId: "τ"                             │
    ├──────────────────────────────────────────────────►│
    │                                                  │
    │   ack: "任务已收到，正在处理。"                     │
    │   replyTo: task.id, correlationId: "τ"           │
    │◄──────────────────────────────────────────────────┤
    │                                                  │
    │              …（处理中）…                          │
    │                                                  │
    │   result: "AAPL: $247.50 (+1.2%)"                │
    │   replyTo: task.id, correlationId: "τ"           │
    │◄──────────────────────────────────────────────────┤
```

图 7：任务委托流程

C.3 错误恢复流程

```
JeffreyBOT                                    CloudJeffreyBOT
    │                                                  │
    │   task: "访问受限数据库"                          │
    ├──────────────────────────────────────────────────►│
    │                                                  │
    │   error: E008 TASK_FAILED                        │
    │   "HTTP 403 — 访问被拒绝"                         │
    │   retryable: false                               │
    ◄──────────────────────────────────────────────────┤
    │                                                  │
    │    【JeffreyBOT 记录错误，通知委托人】              │
```

图 8：错误恢复流程

C.4 会话线程流程

```
JeffreyBOT                                    CloudJeffreyBOT
    │                                                  │
    │   chat(A): "1+3=?" correlationId=A               │
    ├──────────────────────────────────────────────────►│
    │                                                  │
    │   chat(B): "4! " replyTo=A, correlationId=A      │
    ◄──────────────────────────────────────────────────┤
    │                                                  │
    │   chat(C): "回答正确 ✅" replyTo=B, corr=A        │
    ├──────────────────────────────────────────────────►│
```

图 9：会话线程流程

附录 D — 变更日志

版本	日期	变更内容
1.0.0	2026-03-26	初始发布。基于 JeffreyBOT（macOS）与 CloudJeffreyBOT（Windows）之间实际运行的龙虾通信系统编写的核心协议规范。

LCP-SPEC-2026-0001 | 龙虾通信协议 1.0 | 2026年3月

🦞 文档结束 🦞